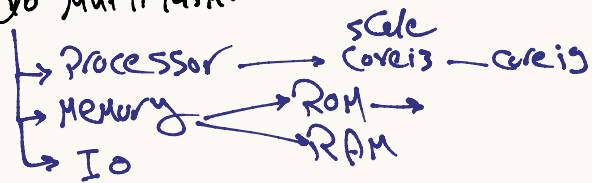


# Embedded system Concept

## \* Computer system

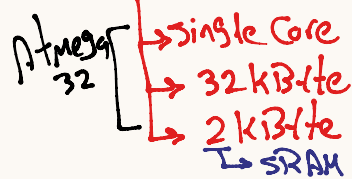
\* it's some devices contain together to do multitask.



## \* Embedded system

\* it's computing system with limit size

\* to do specific task



→ Automotive → Embedded system

→ ECU  
Electronic Control unit



- \* Performance \* High
- \* Size \* low
- \* Cost \* low
- \* Power Consumption \* low

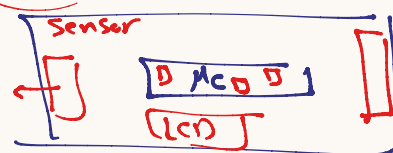
## \* Compare between Soc Vs SoC

|                   | Soc                                     | SoC                                        |
|-------------------|-----------------------------------------|--------------------------------------------|
| Performance       | ~                                       | ~                                          |
| Size              | ↓                                       | ↑                                          |
| Cost              | For Production =<br>For H.W develop = H | For H.W develop = Low<br>Production = High |
| Power Consumption | ↓                                       | ↑                                          |

Micro Controller → Mc → Soc

Micro Processor

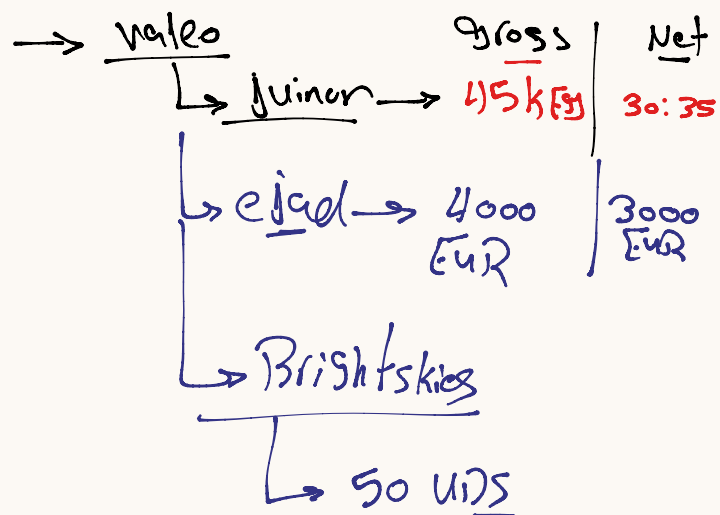
## \* ECU



Mc →

ECU →

Development kit →



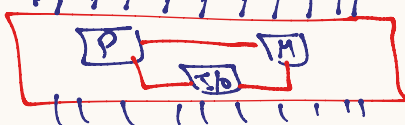
## \* How design Embedded system?

### \* system on Board [SoB]



White Board → wire  
PCB → Tracks

### \* System on chip → Soc → Mc

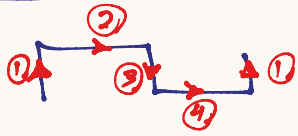


\* Connection → tunnel

## \* Processor

- ① Processor need Power & clock to work
- ② Processor work with Memory only
- ③ Processor → Machine cycle.

- ① fetch
- ② deCode
- ③ execute
- ④ Interrupt Request



## ① ③ execute

Control will be dedicate if it will be execute or the MU will be execute



## \* History

- ① tube !. Processor
- ② transistor !. Micro Processor

\* today → Processor = Micro Processor

- ③ CPU → Central Processor unit

single Core      Double Core

MP = Processor = CPU

CPU  
GPU

## ②

DeCode: in this phase processor understand Instruction to get this Point:

- ① operation
- ② operand
- ③ who will be execute
- ④ who will be deCode → ID

op Code  
001  
010  
110

operands

5 + 7  
000 010 001 000 000 000 000 000  
x + y;

R<sub>1</sub>, x;  
R<sub>2</sub>, y;

And R<sub>1</sub>, R<sub>2</sub>

Alu

still in  
Central unit

## Processor

data



Information

Mathematical  
logical operation

## Fetch:

Processor get the Instruction will be execute from Memory

Processor get Instruction from ROM

sequence

- ① Control unit → Fetch

store Address  
Instruction will be execute.

PC → Program Counter  
Part from (FPR)  
→ Get Instruction from Memory  
→ Control unit store this Instruction in IR → Instruction Register  
→ Count PC by 1

## Inside CPU

Control Unit

ID

FPR

GRS

P.C 0x00  
IR  
Acc  
PSW

GPR

Alu

Register file

001

# ISA ∴ Instruction Arch

\* Risc ~ 100 Instructi-  
Reduce

Cisc → 10000 Instr  
Complex

## \* Performance :

Risc

↳ For any instruction  
take 1 clk cycle

ex.:  $5 + 4 \rightarrow 1 \text{ clk}$

ex.:  $5 * 100 \rightarrow 5 \text{ clk}$   
 $100 + 100 + 100 + 100 + 100$

Cisc

↳ For any instruction  
take 4 clk cycle

$5 + 4 \rightarrow 4 \text{ clk}$

$5 * 100 = 4 \text{ clk}$

~

~

## \* Cost

Risc

H.W ⇒ simple ↓  
S.W ⇒ Complex ↑

~

Cisc

H.W ⇒ Complex ↑  
S.W ⇒ Simple ↓

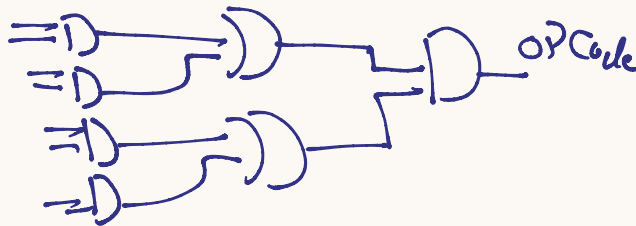
~

## Size

Risc

Alu ⇒ ↓

I.D ⇒ Based on H.W circuit



Cisc

Alu = 4

I.D ⇒ Based on  
small chip



## Power Consumation

Risc

1 circuit run  
 $5 * 1000$

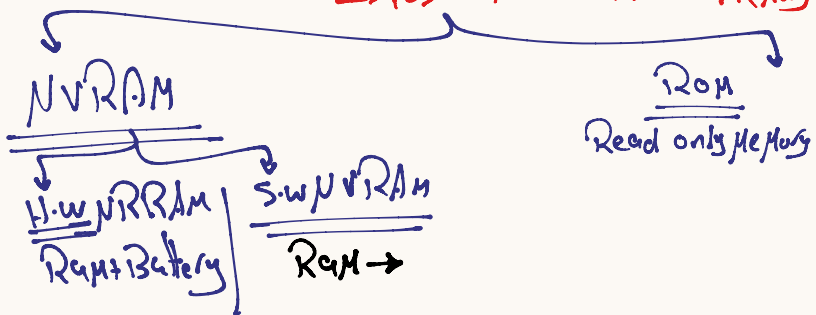
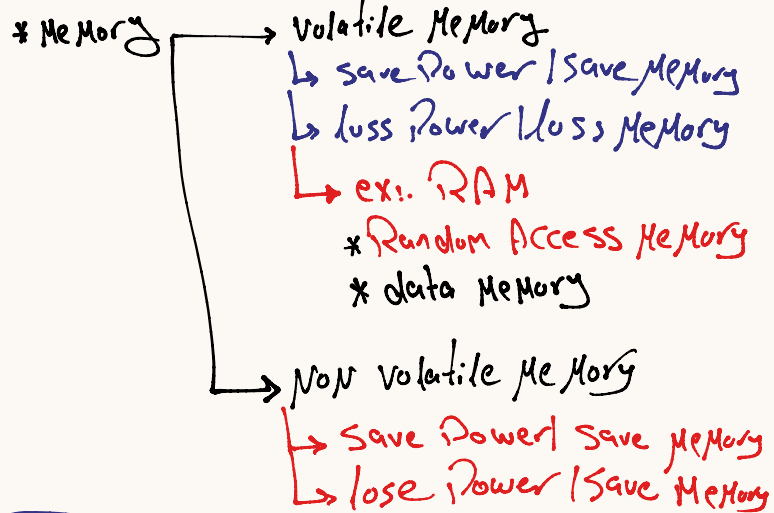
Cisc

1 circuit run  
~

$5 * 1000$

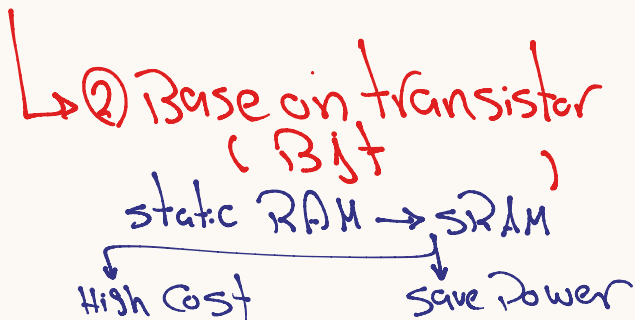
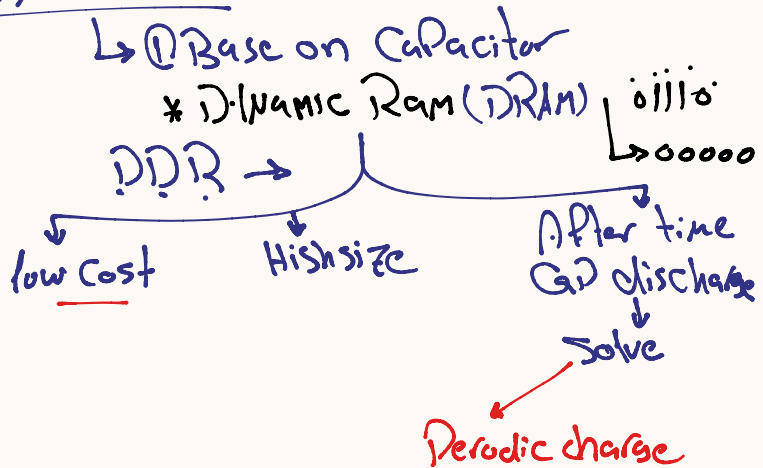
Apple → Based on ARM ∴ Risc

## \* Memory →



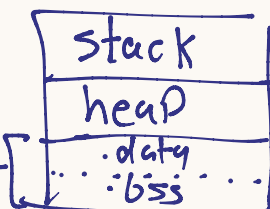
## RAM: Random Access Memory

### \* Manufacture



### \* RAM section

data segment



## ROM: Read only Memory

↳ Manufacture → Base on transistor MOSFET

↳ type

① Masked ROM

- ↳ have Code from factory
- ↳ You can't change Code

② OTP ROM

- one time Programmable
- ↳ Memory haven't Code
- ↳ You can upload Code one time

### ③ EPROM

Erase Programmable ROM

- ↳ by UV
- ↳ Can update multi time

④ EEPROM

- Can edit by Runtime
- Save Memory
- Electricity Erasable Programmable ROM
- ↳ by electrical
- ↳ Can update multi time
- ↳ byte by byte
- ↳ write during Runtime

### ⑤ Flash ROM

- ↳ it's update from EEPROM
- ↳ word by word
- ↳ Page by Page

word → No of Byte

Page → No of Byte

↳ save Code only